



# Stellar Bootcamp

Stellar & Soroban Bootcamp

Prof. Josiel Maimone de Figueiredo

Maio/2026

# Our Bootcamp Classes

**Class 1** (05/05) — Stellar Fundamentals

**Class 2** (12/05) — Custom tokens and Claimable Balances

**Class 3** (19/05) — Smart Contracts with Soroban

**Class 4** (26/05) — Integration and Final dApp

**Class 1** (05/05)

Stellar Fundamentals

**REVIEW**

# Class 1 – Stellar Fundamentals

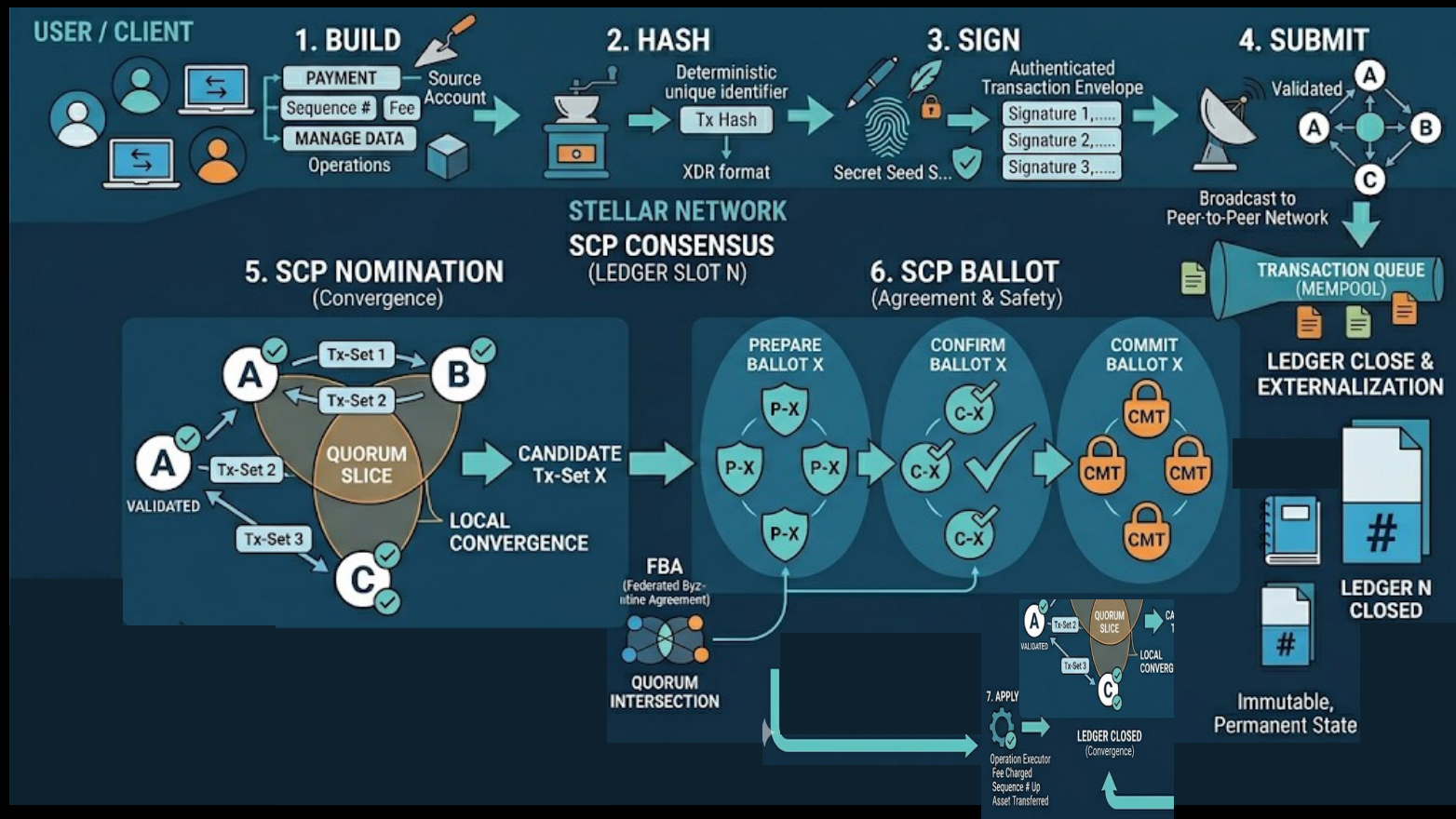
## Your “first” step on Stellar

- Purpose and vision of the Stellar blockchain.
- Differences between Stellar and other blockchains.
- Accounts, public keys, and private keys.
- Testnet and Mainnet.
- Environment setup with Stellar.

## Transactions and security

- Transaction structure on Stellar.
- Cryptographic signatures and account control.
- Simulation of Soroban transactions.
- Complete flow: construction, signing, simulation, and submission

# Stellar Fundamentals



## Class 2 (12/05)

Custom tokens and Claimable Balances

REVIEW

# Class 2 – Custom tokens and Claimable Balances

## Baguncinha journey

- Baguncinha transactions
- Stellar RPC journey

## Baguncinha token

- Smart token
- Claimable clawback

# **Class 3** (19/05)

Smart Contracts with Soroban

**REVIEW**



# Class 3 – Smart Contracts with Soroban

## Rust ultrafast journey

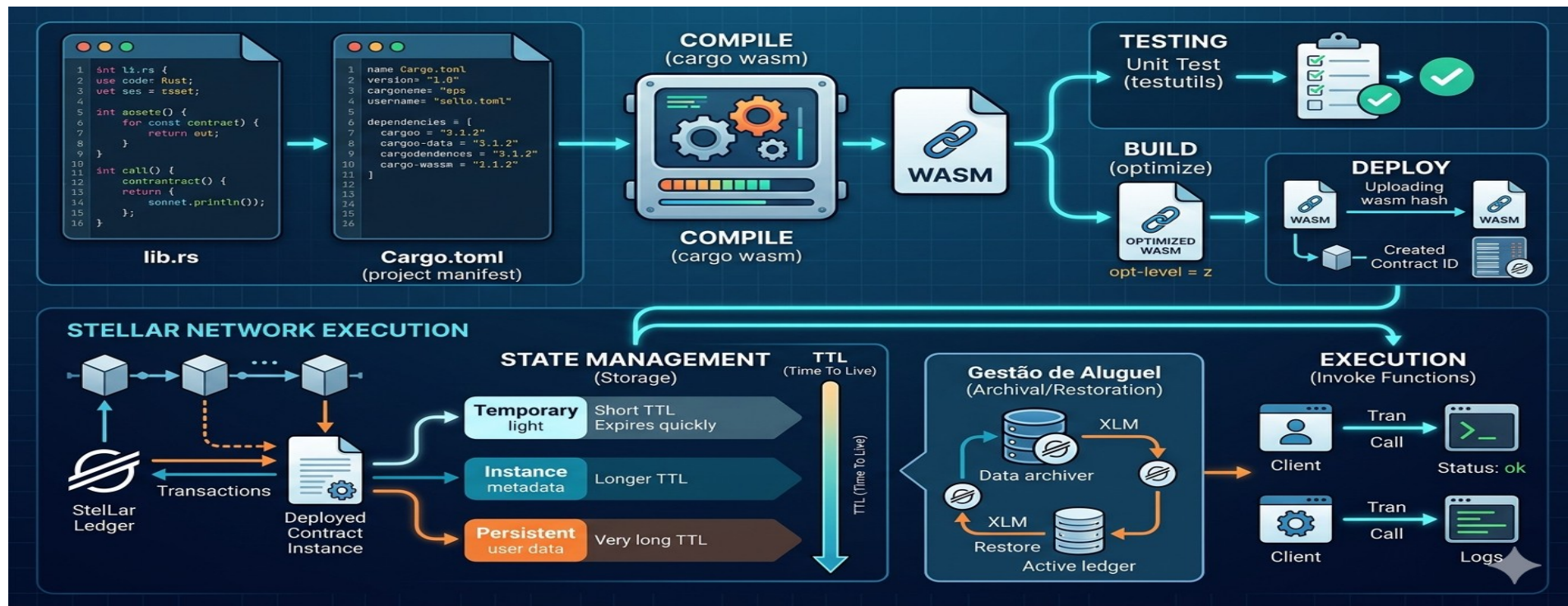
- Rust fundamentals for Baguncinha transactions

## Smart contracts in Soroban

- Smart contract lifecycle for Baguncinha

# Stellar & Rust & Contracts

## Contract lifecycle:



# Class 4 – Integration and final dApp

Before coffee:

## dApp concepts

- Concepts and buzzwords
- Web2 x Web3 – architecture and applications
- Setup environment

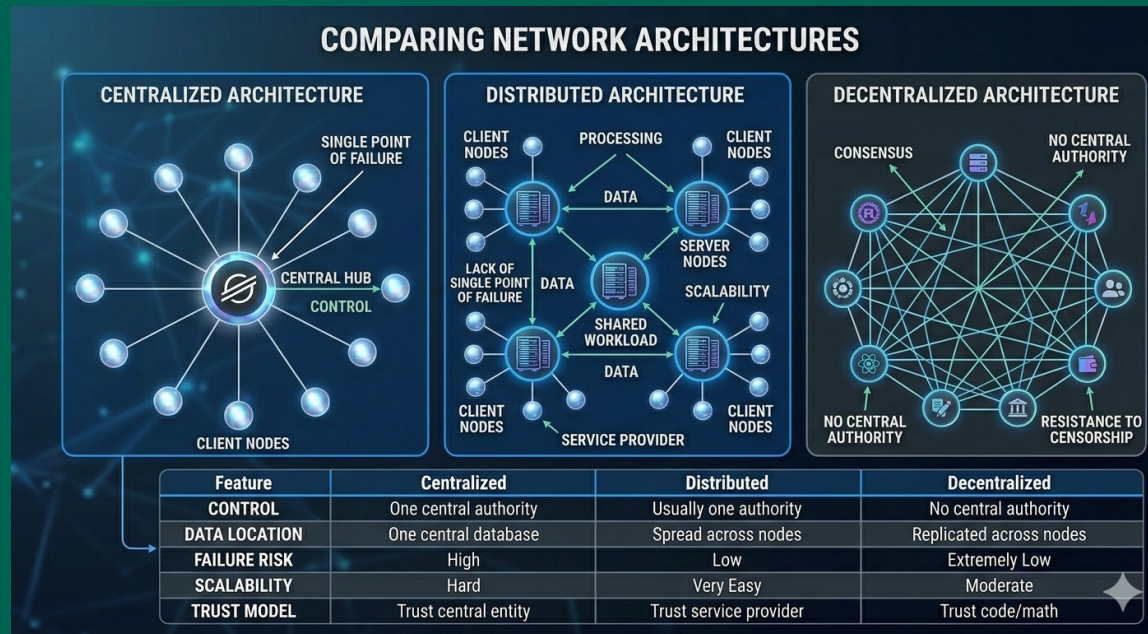
After coffee:

## Final dApp

- Implementation of new dApp
- Baguncinha dApp end journey

# Stellar & dApp

## What is a dApp (Decentralized Application)?



# Stellar & dApp

What is a dApp(Decentralized Application)?



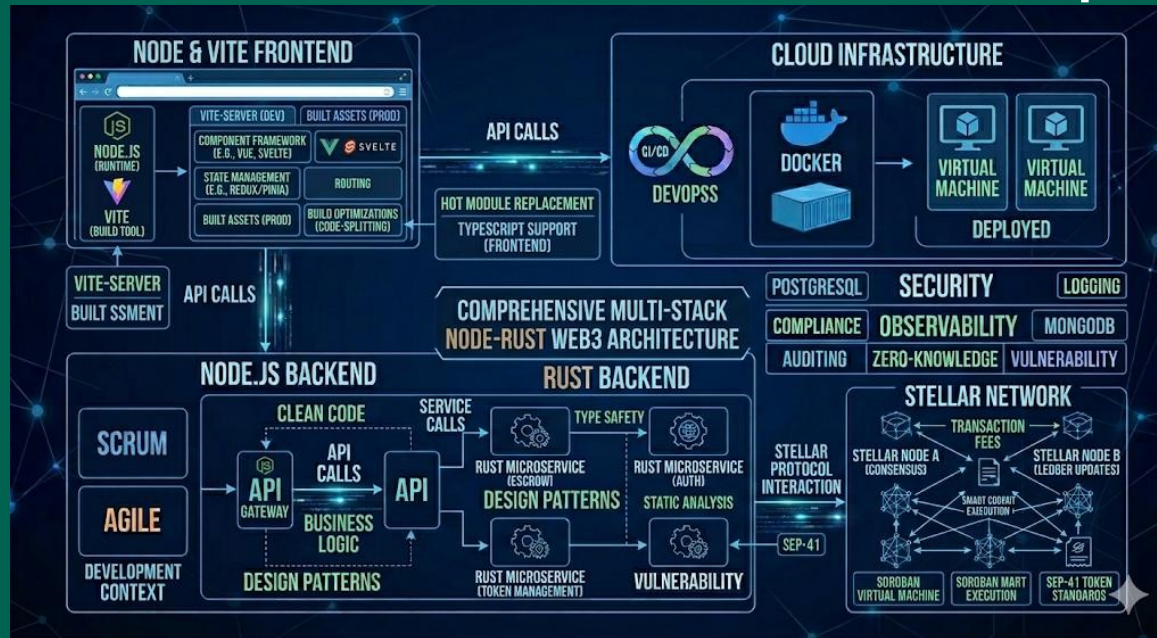


# Buzzwords clouds



# Stellar & dApp

## General architecture and components



# Stellar & dApp

## Setup environment for Rust:

### 1) Rust toolchain:

- **rustup**: version manager and installer.
- **rustc**: Rust compiler.
- **cargo**: project and dependencies.
- **rustfmt**: code formater.
- **clippy**: linter for Rust.

### Code:

```
../myprojects$rustc --version
../myprojects$cargo --version
../myprojects$cargo new mystand
../myprojects$cd mystand
../myprojects/mystand$cargo run
```



# Stellar & dApp

## Setup environment for Stellar:

2) Choose an IDE with:

- Rust toolchain
- CLI Stellar
  - Latest version: 26.0.0
- <https://developers.stellar.org/docs/build/smart-contracts/getting-started/setup>

Code:

```
$ cargo install --locked stellar-cli
```

```
Compiling indexmap v1.1.0
Compiling crypto-common v0.1.6
Compiling block-buffer v0.10.4
Compiling data-encoding v2.9.0
Compiling clap_lex v0.7.5
Compiling clap_builder v4.5.47
Compiling digest v0.10.7
Compiling serde_derive v1.0.228
Compiling zeroize_derive v1.4.2
Compiling thiserror-impl v1.0.69
Compiling futures-macro v0.3.31
Compiling zeroize v1.8.1
Compiling darling_macro v0.20.11
Compiling tokio-macros v2.5.0
Compiling darling v0.20.11
Compiling serde_with_macros v3.14.0
Compiling futures-util v0.3.31
Compiling tokio v1.47.1
Compiling sha2 v0.10.9
Building [====>] 139/694:
```

# Stellar & dApp

## Test environment for Stellar:

### 3) Test contract lifecycle:

- Init soroban project
- Generate keys for Jão
- Build contract
- Deploy contract
- Invoke contract

### Code:

```
$rustup target add wasm32-unknown-unknown
$rustup target add wasm32v1-none
$cargo add soroban_sdk
$stellar contract init mystand
$cargo test

Put in Cargo.toml:
[profile.release]
overflow-checks = true

$stellar contract build
$stellar contract deploy --wasm
target/wasm32v1-none/release/mybagu.wasm \
--source-account jao --network testnet --alias mybagu
```

# Stellar & dApp

## Test environment for Stellar:

### 3) Test contract lifecycle:

- Generate keys for Jão
- Build contract
- Init contract
- Deploy contract
- Invoke contract

### Code:

```
$stellar contract invoke  
  --id CACDY...SOROBAN  
  --source-account jao  
  --network testnet  
  -- mymethod --par1 val1 --par2 val2
```

# Stellar & dApp

## Frontend Stellar:

### 4) Node install

- Install node
- Install Typescript
- Init dapp project
  - install stellar-sdk on project
  - install wallet connection on project
- <https://github.com/nvm-sh/nvm#installing-and-updating>

### Code:

```
$curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.4/install.sh | bash
$nvm install node

$npm install -g typescript tsx

$mkdir mystand-dapp && cd mystand-dapp
$npm init -y
# Install Stellar SDK JS
$npm install @stellar/stellar-sdk
# Install Freighter API
$npm install @stellar/freighter-api
# Verify SDK installation
$node -e "const s = require('@stellar/stellar-sdk');
console.log(s.Networks.TESTNET) "
Test SDF Network ; September 2015
```

# Stellar & dApp

## Frontend Stellar:

### 5) Verify installations

- Install node
- Install Typescript
- Init dapp project
  - install stellar-sdk on project
  - install wallet connection on project
- Verify  
<https://github.com/nvm-sh/nvm#installing-and-updating>

### Code:

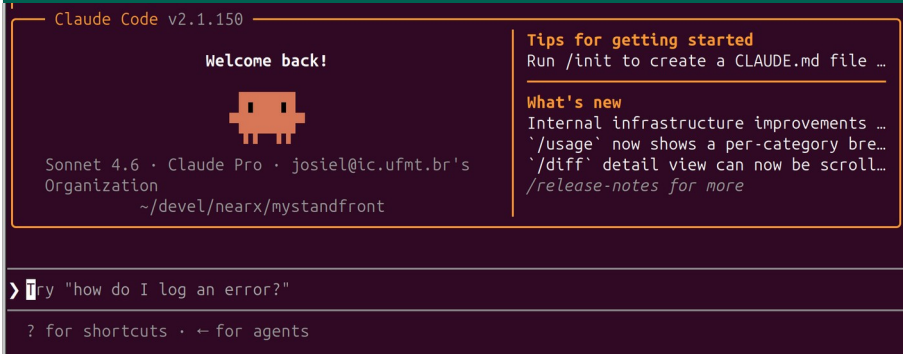
```
$nvm install node
echo "=== Versions for Stellar DApp tools==="
echo "Rust:"
rustc --version && cargo --version
echo "WASM target:"
rustup target list --installed | grep wasm32
echo "Stellar CLI:"
stellar --version
echo "Node.js:"
node --version && npm --version
echo "TypeScript:"
tsc --version
echo "Stellar SDK:"
node -e "const s=require('@stellar/stellar-sdk');
        console.log('SDK OK -', s.Networks.TESTNET)"
echo "=== Environment ready ==="
```

# Stellar & dApp

## Claude Code with Stellar:

### 6) Claude Code CLI

- **Configure claude cli**



```
Claude Code v2.1.150

Welcome back!

[Pixelated Robot]

Sonnet 4.6 · Claude Pro · josiel@ic.ufmt.br's
Organization
~/devel/nearx/mystandfront

> Try "how do I log an error?"

? for shortcuts · +- for agents
```

Code:

```
$npm install -g @anthropic-ai/claude-code
$claude --version
$which claude
```

# Stellar & dApp

## Claude Code with Stellar:

### 7) Claude Code CLI

- Configure claude cli
  - Enhance answers with CLAUDE.md file
  - Create permissions file

Code:

```
cat > CLAUDE.md << 'EOF'
# Stellar DApp - Project context

## Tech Stack
- Contracts: Rust 1.80+ + Soroban SDK 21
- Frontend: React 18 + TypeScript 5 + Vite + Tailwind
- Blockchain: Stellar Testnet
- Wallet: Freightier Wallet
- SDK JS: @stellar/stellar-sdk v12+

## Usual commands
- Build:    cd contracts/token && stellar contract build
- Test:     cd contracts/token && cargo test
- Deploy:   stellar contract deploy --network testnet
- Frontend: cd frontend && npm run dev

## Net and endpoints
- RPC Testnet: https://soroban-testnet.stellar.org
- Explorer:   https://stellar.expert/explorer/testnet
- Friendbot:  https://friendbot.stellar.org
- CONTRACT_ID: see frontend/src/config.ts

## Conventions
- Rust: snake_case for functions
- React: PascalCase for components
- Hooks: prefix use (useWallet, useContract)
- Env vars frontend: prefix VITE_

## Security
- Never commit .env with secret keys
- Always use require_auth() in mutant functions
- Review contracts before deploy on mainnet
EOF
```



# Stellar & dApp

## Claude Code with Stellar:

### 7) Claude Code CLI

- Configure claude cli
  - Enhance answers with CLAUDE.md file
  - **Create permissions file, 3 levels:**
    - allow
    - (not listed)
    - deny

Code:

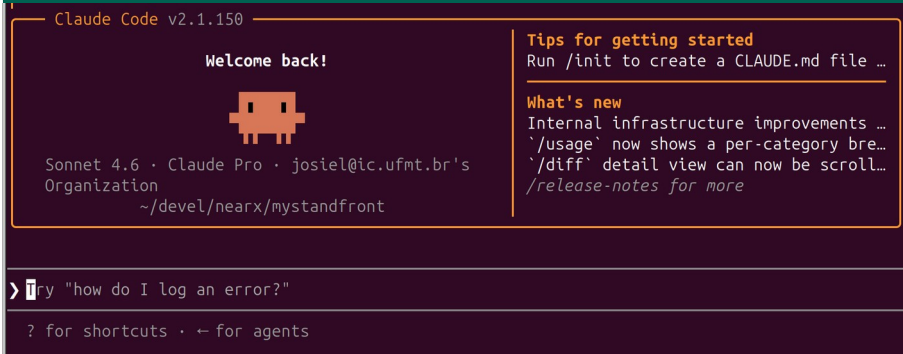
```
mkdir -p .claude
cat > .claude/settings.json << 'EOF'
{
  "permissions": {
    "allow": [
      "Bash(cargo build*)",
      "Bash(cargo test*)",
      "Bash(cargo check*)",
      "Bash(cargo clippy*)",
      "Bash(stellar contract build*)",
      "Bash(stellar contract invoke --network testnet*)",
      "Bash(npm run*)",
      "Bash(npm install*)",
      "Bash(git status)",
      "Bash(git diff*)",
      "Write(contracts/**)",
      "Write(frontend/src/**)"
    ],
    "deny": [
      "Bash(stellar contract deploy --network mainnet*)",
      "Bash(rm -rf*)",
      "Bash(git push --force*)",
      "Write(.env*)"
    ]
  }
}
EOF
```

# Stellar & dApp

## Claude Code with Stellar:

### 6) Claude Code CLI

- **Connect to claude cli**



Code:

```
$cd mystand-dapp && claude
Order to claude:
"Create the folder structure: contracts, frontend,
scripts"
"Create the Cargo.toml workspace including contracts/"
"Which version of the soroban-sdk is in Cargo.toml?"
"Generates tests for contracts/token/src/lib.rs"
"Add new folders and files to git. Analyze the diff and
write a commit message."
"Create the Cargo.toml workspace in the root directory
with members=["contracts/token"] and edition=2021."
"Creates contracts/token/Cargo.toml with soroban-sdk 21
and contracts/token/src/lib.rs with minimal boilerplate."
"Run cargo on the root directory and tell me if there are
any errors."
"Create a .gitignore file for Rust + Node and a basic
README.md file."
```

# Stellar & dApp

## Claude Code & GIT with Stellar:

### 8) Claude Code CLI & GIT

- Connect to claude cli
- **Connect to git**
  - Create git repository with gh
  - Initialize
  - Add, commit and push files

### Code:

```
$gh repo create mystand-dapp --public --description "DApp in
Stellar with Soroban"
$cd mystand-dapp
$git init
$git branch -M main
$git remote set-url origin https://github.com/USER/mystand-
dapp.git
$git remote -v
$git add .
$git status
$git commit -m "feat: starting our Stellar Dapp project"
$git push -u origin main
```

# Stellar & dApp

## Claude Code & GIT with Stellar:

### 8) Claude Code CLI & GIT

- Connect to claude cli
- Connect to git
  - Review .gitignore

Code:

```
cat > .gitignore << 'EOF'
# ===== RUST / SOROBAN =====
/target/
**/*.wasm
Cargo.lock

# ===== NODE / FRONTEND =====
node_modules/
dist/
.next/
build/
*.tsbuildinfo

.env
.env.local
.env*.local
.env.production

# ===== STELLAR / SOROBAN CLI =====
.stellar/

# ===== EDITORS / OS =====
.DS_Store
.vscode/settings.json
.idea/
*.swp
*.swo
EOF
```

# Stellar & dApp

## Claude Code & GIT with Stellar:

### 8) Claude Code CLI & GIT

- Connect to claude cli
- Connect to git
  - Create file .gitignore
  - Create project branch structure
- **Configure Branch Protection Rules on GitHub**

### Code:

```
$git checkout -b develop
$git push -u origin develop
$git checkout develop
$git checkout -b feature/contract-token
$git add contracts/token/src/lib.rs
$git commit -m "feat(contract): add mint function with
require_auth"
$git push origin feature/contract-token
```

# Stellar & dApp

## Claude Code & GIT with Stellar:

### 8) Claude Code CLI & GIT

- Connect to claude cli
- Connect to git
  - Create file .gitignore
  - Create project branch structure
- **Configure on GitHub:**
  - **Branch Protection Rules**
  - **Secrets key** → new key from Stellar

### Code:

Access on browser: `github.com/usuario/stellar-dapp`

→ **Settings** → **Branches** → **Add rule**

- Branch name pattern: **main**
- Set: **Require a pull request before merging**
- Set: **Require status checks to pass**
- Set: **Do not allow bypassing the above settings**

Add GitHub Secrets for CI/CD:

**Settings** → **Secrets and variables** → **Actions** → **New repository secret**

**STELLAR\_SECRET\_KEY** – private key S... from deploy account on Testnet. Never use account with real funds.

**VITE\_CONTRACT\_ID** – contract ID on Testnet, updated by workflow after each deploy

# Stellar & dApp

## Claude Code & GIT with Stellar:

### 8) Claude Code CLI & GIT

- Connect to claude cli
- Connect to git
- Configure on GitHub
  - **CI/CD Workflow on GitHub**

```
Code:
mkdir -p .github/workflows
cat > .github/workflows/ci.yml << 'EOF'
name: CI - Build & Test Soroban

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main, develop ]

env:
  CARGO_TERM_COLOR: always

jobs:
  build-and-test:
    name: Build and Test
    runs-on: ubuntu-latest
    steps:
      - name: Repository Checkout
        uses: actions/checkout@v4
      - name: Install Rust and target WASM
        uses: dtolnay/rust-toolchain@stable
        with:
          targets: wasm32-unknown-unknown
      - name: Cargo cache
        uses: Swatinem/rust-cache@v2
        with:
          workspaces: ". -> target"
      - name: Install Stellar CLI
        run: cargo install --locked stellar-cli --features opt
      - name: Contract Build (WASM)
        run: stellar contract build
        working-directory: contracts/token
      - name: Unit tests
        run: cargo test --verbose
        working-directory: contracts/token
      - name: Clippy - check warnings
        run: cargo clippy -- -D warnings
        working-directory: contracts/token
EOF
```

# Stellar & dApp

## Claude Code & GIT with **Stellar**:

### 8) Claude Code CLI & GIT

- Connect to claude cli
- Connect to git
- Configure on GitHub
  - **CI/CD Workflow on GitHub**

Code:

```
$git add .github/workflows/ci.yml  
$git commit -m "ci: add workflow"  
$git push origin develop
```



# Stellar & dApp

## Claude Code & Stellar:

### 9) Contracts

- Stellar is on?
- Generate new keys
- Show key public address
- Fund new account
- Show account data
- Show last 5 transaction on account
- Show a transaction confirmation
- View ledger

Code:

```
$stellar network ls
$stellar network use testnet
$stellar keys generate dev-key --network testnet
$stellar keys address dev-key
$stellar keys fund dev-key --network testnet
$curl -s
"https://horizon-testnet.stellar.org/accounts/<PUBLIC_KEY>" | jq
'.balances'
$curl -s
"https://horizon-testnet.stellar.org/accounts/<PUBLIC_KEY>" |
python3 -m json.tool | head -40
$curl -s
"https://horizon-testnet.stellar.org/accounts/GXXX.../transactions?limit=5&order=desc" | python3 -m json.tool | grep -E
'"hash"|"created_at"|"operation_count"'
$curl -s "https://horizon-testnet.stellar.org/transactions/HASH"
| python3 -m json.tool | grep -E '"successful"|"fee_charged"'
$curl -s "https://horizon-testnet.stellar.org/ledgers?
order=desc&limit=1" | python3 -m json.tool | grep -E
'"sequence"|"closed_at"|"transaction_count"'
```

# Stellar & dApp

## Claude Code & Stellar:

### 9) Contracts

- Implement contract

Code contracts/token/src/lib.rs:

```
#![no_std]
use soroban_sdk::{
    contract, contractimpl, contracttype,
    contracterror, token,
    symbol_short, Address, Env, String, BytesN,};

#[contracttype]
#[derive(Clone, PartialEq, Eq)]
pub enum DataKey {
    Admin, StandName, BaguncinhasLeft,
    TotalSold, V2Migrated, TokenAddress, }

#[contracterror]
#[derive(Clone, Copy, Debug, PartialEq, Eq)]
pub enum StandError {
    NotAdmin = 101,
    InsufficientStock = 202,}
```

# Stellar & dApp

## Claude Code & Stellar:

### 9) Contracts

- Implement contract

```
Code contracts/token/src/lib.rs:

#[contracttype]
#[derive(Clone, Debug)]
pub struct SaleEvent { pub customer: Address, pub quantity: u32, pub total_paid: i128, pub order_id: u32, }
#[contract]
pub struct Baguncinha;
#[contractimpl]
impl Baguncinha {
    // Initialization
    pub fn init(env: Env, admin: Address, name: String, token_address: Address) {
        if env.storage().instance().has(&DataKey::Admin) {
            panic!("Already initialized");
        }
        env.storage().instance().set(&DataKey::Admin, &admin);
        env.storage().instance().set(&DataKey::StandName, &name);
        env.storage().instance().set(&DataKey::TokenAddress, &token_address);
        env.storage().persistent().set(&DataKey::BaguncinhasLeft, &0u32);
        env.storage().persistent().set(&DataKey::TotalSold, &0u32);
        env.storage().instance().set(&DataKey::V2Migrated, &false);
    }

    // Optimized Sell
    pub fn buy_and_earn( env: Env, customer: Address, quantity: u32,
    ) -> Result<u32, StandError> {
        customer.require_auth();
        let mut stock = env.storage().persistent().get(&DataKey::BaguncinhasLeft).unwrap_or(0);
        let mut total_sold = env.storage().persistent().get(&DataKey::TotalSold).unwrap_or(0);
        if stock < quantity {
            return Err(StandError::InsufficientStock);
        }
        let token_address: Address = env.storage().instance().get(&DataKey::TokenAddress).unwrap();
        let price = (quantity as i128) * 100_000_000;
        let token_client = token::Client::new(&env, &token_address);
        token_client.transfer(&customer, &env.current_contract_address(), &price);
        stock -= quantity;
        total_sold += quantity as u32; // ← CORRIGIDO
        env.storage().persistent().set(&DataKey::BaguncinhasLeft, &stock);
        env.storage().persistent().set(&DataKey::TotalSold, &total_sold);
        env.storage().persistent().extend_ttl(&DataKey::BaguncinhasLeft, 17280, 120960);
        env.storage().persistent().extend_ttl(&DataKey::TotalSold, 17280, 120960);
    }
}
```

# Stellar & dApp

## Claude Code & Stellar:

### 9) Contracts

- Implement contract

```
Code contracts/token/srclib.rs:

// Emit event (avoid double move)
let customer_clone = customer.clone();
env.events().publish(
    (symbol_short!("sell"), customer_clone),
    SaleEvent {
        customer,
        quantity,
        total_paid: price,
        order_id: total_sold,
    },
);

Ok(total_sold)
}

// Upgrade
pub fn upgrade(env: Env, new_wasm_hash: BytesN<32>) {
    let admin: Address = env.storage().instance().get(&DataKey::Admin).unwrap();
    admin.require_auth();
    env.deployer().update_current_contract_wasm(new_wasm_hash);
}

// Helper: Add stock (admin only)
pub fn add_stock(env: Env, quantity: u32) -> u32 {
    let admin: Address = env.storage().instance().get(&DataKey::Admin).unwrap();
    admin.require_auth();

    let mut stock = env.storage().persistent().get(&DataKey::BaguncinhasLeft).unwrap_or(0);
    stock += quantity;
    env.storage().persistent().set(&DataKey::BaguncinhasLeft, &stock);
    stock
}
}
```

# Stellar & dApp

## Claude Code & Stellar:

### 9) Contracts

- Implement contract
- Build contract → generate wasm
- Deploy contract
- Invoke contract
  - Init method

Code :

```
Order claude:
"build contract, generate wasm and
deploy contract"
To invoke init:
-Admin:
"GDL5FSNTT7NAAXUQ2GI4GVLWDZ5SUB5VLJNA6
IOZT4RHNEZEVTC7QHFU"
- name:"Bagunça"
```

# Stellar & dApp

## Claude Code & Frontend with Stellar:

### 9) Claude Code CLI & Frontend

- Order claude to create frontend and its infrastructure

Code → on Claude code:

Create componentes Card, Badge and Input in src/components/ui/ following the same standard of Button.tsx with the same Stellar color palette.

```
frontend/src/  
├ components/  
│ └ ui/  
│   └ wallet/  
│     └ token/  
├ hooks/  
├ lib/  
├ pages/  
├ config.ts  
├ App.tsx  
└ main.tsx
```

# Stellar & dApp

## Claude Code & Frontend with Stellar:

### 10) Claude Skills

- On Claude → **skill to deploy the contract**

Code → generate Stellar skill:

```
mkdir -p .claude/skills
cat > .claude/skills/deploy-soroban.md << SKILL
# Skill: Deploy Soroban Contract
## Triggers
- "deploy the contract"
- "deploy on testnet"
- "publish token"
- "update contract on testnet"
## Context
Build, deploy on Stellar Testnet and update
CONTRACT_ID in all project files.
Use dev-key identity from Stellar CLI.
## Steps
1. Navigate to contracts/sep41_token/
2. Execute: stellar contract build
3. Verify that .wasm was generated
4. Execute: stellar contract deploy --wasm ...
5. Capture the returned CONTRACT_ID
6. Update frontend/src/config.ts with new ID
7. Update CLAUDE.md in the section CONTRACT_ID
8. Confirm with: stellar contract invoke --id ID -- name
## Commands
```bash
cd contracts/sep41_token
stellar contract build
stellar contract deploy \
  --wasm target/wasm32-unknown-unknown/release/sep41_token.wasm \
  --source dev-key \
  --network testnet
...
## In error
- "account not found": execute stellar keys fund dev-key --network testnet
- "wasm not found": verify that stellar contract build has passed
SKILL
```

# Stellar & dApp

## Claude Code & Frontend with Stellar:

### 10) Claude Skills

- On Claude → generate test to the contract 123

Code → generate Test skill:

```
cat > .claude/skills/generate-tests.md << SKILL
# Skill: Generate Soroban Tests

## Triggers
- "generate tests for the contract"
- "create unit tests"
- "add tests"

## Context
Generate complete unit tests for Soroban contracts
using soroban_sdk testutils, Env::default() and mock_all_auths.
Always cover: happy path, authorization fault,
no balance and generated events verification.

## Steps
1. Read the lib.rs file of the specified contract
2. Identify all public functions
3. For each mutant function: generate authentication success and failure tests
4. For value functions: generate insufficient balance test
5. For functions with events: generate event verification test
6. Write to src/test.rs
7. Run load test and automatically correct errors

## Mandatory structure for each test
```rust
#[test]
fn test_function_name() {
    let env = Env::default();
    env.mock_all_auths();
    let contract_id = env.register(MyContract, ());
    let client = MyContractClient::new(&env, &contract_id);
    // arrange, act, assert
}
...
SKILL
```



# Stellar & dApp

## Claude Code & Frontend with Stellar:

### 10) Claude Skills

- On Claude
  - Generate a complete README.md file for the project.
  - Documenting the Soroban contract with Rust comments
  - Generate frontend documentation (JSDoc)

### Code → execute on Claude:

Generates a complete README.md file for the project in the root directory. Includes: description, architecture (ASCII diagram), tech stack, how to run locally, how to deploy, environment variables, folder structure and MIT license.

/////

Add complete documentation to the contract in contracts/sep41\_token/src/lib.rs. For each published function: Description of what it does; Parameters with type and meaning; What it returns; Possible errors; Example of use via stellar contract invoke.

/////

Add JSDoc to all hooks in frontend/src/hooks/. For each hook: description, parameters, return value, and usage example.

TypeScript JSDoc format compatible with VS Code.

# Stellar & dApp

## Claude Code & Frontend with Stellar:

### 10) Complete Git workflow

```
cat > .github/workflows/deploy.yml << WORKFLOW
name: CI e Deploy
on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main, develop]
jobs:
  test-contract:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: dtolnay/rust-toolchain@stable
        with:
          targets: wasm32-unknown-unknown
      - uses: Swatinem/rust-cache@v2
      - name: Instalar Stellar CLI
        run: cargo install --locked stellar-cli --features opt
```

Code → execute on Claude:

```
- name: Build contrato
  run: stellar contract build
  working-directory: contracts/sep41_token
- name: Testes Rust
  run: cargo test --verbose
  working-directory: contracts/sep41_token
- name: Clippy
  run: cargo clippy -- -D warnings
  working-directory: contracts/sep41_token

build-frontend:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-node@v4
      with:
        node-version: 20
        cache: npm
        cache-dependency-path: frontend/package-lock.json
    - run: npm ci
      working-directory: frontend
    - run: npm run build
      working-directory: frontend
    env:
      VITE_TOKEN_ID: ${vars.TOKEN_ID}
  - name: Deploy GitHub Pages
    if: github.ref == "refs/heads/main"
    uses: peaceiris/actions-gh-pages@v4
    with:
      github_token: ${secrets.GITHUB_TOKEN}
      publish_dir: frontend/dist
```

WORKFLOW

# Stellar & dApp

## Claude Code & Frontend with Stellar:

### 10) Complete Git deploy

- Enable GitHub Pages in the repository
- Go to Settings → Pages in the GitHub repository
- Source: Deploy from a branch
  - Branch: gh-pages → / (root)
- Save — GitHub Pages enabled in
  - user.github.io/mystand-dapp
- Add variable:
  - Settings → Variables → Actions → TOKEN\_ID=CXXX...
- Do first deploy

### Code → first deploy:

```
# Merge to main triggers the workflow
automatically
$git checkout main
$git merge develop
$git push origin main
Verify on claude:
"Check if the production deployment is correct:
- vite.config.ts has a base configuration
- frontend/src/config.ts uses the VITE_TOKEN_ID
from the environment
- .github/workflows/deploy.yml passes the
TOKEN_ID correctly
- there are no hardcoded contract IDs in the
code "
```

# Stellar & dApp

## Claude Code & Frontend with Stellar:

### 10) Verify deploy

- Enable GitHub Pages in the repository
- Go to Settings → Pages in the GitHub repository
- Source: Deploy from a branch
  - Branch: gh-pages → / (root)
- Save — GitHub Pages enabled in
  - user.github.io/mystand-dapp
- Add variable:
  - Settings → Variables → Actions → TOKEN\_ID=CXXX...
- Do first deploy

Code → verify deploy:

```
# 1. Skills created
$ls .claude/skills/
$deploy-soroban.md generate-tests.md
# 2. README.md exists and has content
$wc -l README.md
180 README.md
# 3. Rust tests all passing
$cd contracts/sep41_token && cargo test 2>&1 | tail -1
test result: ok. 18 passed; 0 failed
# 4. Build frontend ok
$cd frontend && npm run build 2>&1 | tail -3
dist/index.html 1.4 kB
dist/assets/index.js 312 kB

#5. Green CI Workflow on GitHub
# Check at: github.com/usuario/stellar-dapp/actions
# 6. Online DApp
# Check at: usuario.github.io/stellar-dapp
```

# Stellar & dApp

## Claude Code & Frontend with Stellar:

### 10) Contract security by Claude

- Some security examples

Code → execute on Claude:

```
Perform a complete security audit of the contract

in contracts/sep41_token/. Check:

1. AUTHORIZATION: Does every mutant function have require_auth()?
Are there functions that should be restricted to the admin but are not?

2. OVERFLOW: Are i128 operations protected against overflow?
Does Cargo.toml have overflow-checks = true in the profile release?

3. STORAGE TTL: Does Persistent data have extend_ttl?
Is the TTL sufficient for production use (> 30 days)?

4. INITIALIZE: Does the initialize function prevent duplicate calls?
Can the contract be restarted by an attacker?

5. EVENTS: Do all value operations emit events?
Do Mint, burn, and transfer have events with correct topics?

6. ALLOWANCE EXPIRY: Is the expiry_ledger validated before being used?

Are expired allowances treated as zero or as an error?

For each item: OK / PROBLEM / SUGGESTION FOR IMPROVEMENT
```

# Stellar & dApp

## Claude Code & Frontend with Stellar:

### 10) Contract security by Claude

- Some security examples

### Code → execute on Claude:

For each function published in  
contracts/sep41\_token/src/lib.rs,

```
list: name | type (read/write) | has  
require_auth? | risk
```

Mark as CRITICAL any write without  
require\_auth

# Stellar & dApp

## Claude Code & Frontend with Stellar:

### 10) Deploy on Mainnet

- 7 steps of Mainnet deployment — never fully automate
  - **Final build** - `stellar contract build` on the main branch after green CI.
  - **Verify API** - `stellar contract inspect` confirms exported functions before uploading.
  - **Simulate cost** - `--sim-only` flag shows the exact XML before execution.
  - **Upload WASM** - upload bytecode with explicit manual approval.
  - **Create instance** - instantiate and save `CONTRACT_ID` in a safe place.
  - **Initialize** - call `initialize()` with correct parameters. Irreversible.
  - **Verify in explorer** - confirm everything in Stellar Expert before announcing.

Code → execute on Claude:

```
# 1. Build final
cd contracts/sep41_token && stellar contract build
Finished release [optimized] target(s)

# 2. Verificar API
stellar contract inspect \
  --wasm target/wasm32-unknown-unknown/release/sep41_token.wasm
initialize, mint, burn, transfer, transfer_from, balance, ...

# 3. Simulate costs (--sim-only)
stellar contract deploy \
  --wasm target/wasm32-unknown-unknown/release/sep41_token.wasm \
  --source mainnet-deploy --network mainnet --sim-only
Estimated fee: 1.2 XLM
Estimated storage: 0.5 XLM (locked as base reserve)

# 4. Deploy real (only after simulate costs)
stellar contract deploy \
  --wasm target/wasm32-unknown-unknown/release/sep41_token.wasm \
  --source mainnet-deploy --network mainnet
CXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX

# 5. Initilize (IRREVERSIBLE)
stellar contract invoke \
  --id $MAINNET_TOKEN_ID --source mainnet-deploy --network mainnet \
  -- initialize \
  --admin $(stellar keys address mainnet-deploy) \
  --decimals 7 --name "MeuToken" --symbol "MTK"
null
```



# Stellar Bootcamp

That's all, Folks!!!

“Valeu Xômano!!”